

Contents

Конспект лекции: Асимптотический анализ алгоритмов	1
Организационная информация	1
Лектор	1
Система оценивания	2
1. Что такое алгоритм	2
2. Функция трудоёмкости $T(n)$	3
3. Зачем нужна асимптотическая оценка: пример сравнения двух алгоритмов	3
4. Асимптотические обозначения	4
4.1. Оценка сверху: O («О большое»)	4
4.2. Оценка снизу: Ω («Омега большое»)	5
4.3. Точная асимптотическая оценка: Θ («Тета большое»)	5
5. Примеры вычисления сложности циклов	6
5.1. Цикл с фиксированным числом итераций	6
5.2. Цикл, зависящий от размера входа	6
5.3. Вложенный цикл, где внешний цикл — константный	6
5.4. Цикл с условием, зависящим от квадрата счётчика	6
6. Свойства асимптотических обозначений	7
6.1. Транзитивность	7
6.2. Рефлексивность	7
6.3. Симметрия	7
7. Сортировка вставками (Insertion Sort)	7
7.1. Вывод оценки сложности	8
7.2. Почему константы отбрасываются	8
7.3. Границы (лучший и худший случай)	9
7.4. Оценка по памяти	9

Конспект лекции: Асимптотический анализ алгоритмов

Организационная информация

Лектор

Ткаченко Даниил Михайлович

- Более 8 лет работает в ИТ.
- Более 7 лет преподаёт.
- Опыт работы: разработчик, руководитель, аналитик. Сейчас — системный архитектор.

Важно: лекции, практики и лабораторные работы должны быть качественными. Если что-то не так — жаловаться лектору. Подробная информация о курсе — в `unote`. С помощью первой лабораторной работы можно познакомиться с проверяющей системой.

Система оценивания

Всего за курс можно набрать **100 баллов**. Все баллы набираются в течение семестра, отдельного экзамена в привычном смысле нет (см. ниже про допуск к экзамену).

№	Компонент	Баллы	Комментарий
1	Лабораторные работы	54	Итоговый балл = (ваши баллы / максимум баллов) × 54
2	Тесты	18	—
3	Live Coding	до 8	—
4	Командный коллоквиум	10	Первый коллоквиум — командный, команда из 4-6 человек
5	Теормин	10	Нужно набрать минимум 8 баллов , даётся 2 попытки

Правила, связанные с теорминимумом и допуском:

- Если теормин не сдан за 2 попытки — студент направляется на дополнительную сессию (допсу).
- Если к концу семестра набрано от **40 до 60 баллов** и теормин сдан — можно прийти на экзамен.
- Экзамен оценивается по системе «сдал / не сдал» (без дифференцированной оценки).

1. Что такое алгоритм

Алгоритм — это способ преобразования входных данных (ввода) в выходные данные (вывод) с помощью определённой последовательности действий.

Чтобы можно было анализировать алгоритм, его действия принято раскладывать на **атомарные операции** — элементарные шаги, каждый из которых считается занимающим одну «единицу работы»:

- сложение;
- вычитание;
- умножение;
- деление;
- одна итерация цикла (проверка условия + переход к следующему шагу).

Считается, что каждая такая операция выполняется за постоянное время, не зависящее от размера входных данных.

2. Функция трудоёмкости $T(n)$

Чтобы оценить, «насколько дорогой» алгоритм, вводится функция $T(n)$ — количество атомарных операций, которое выполняет алгоритм в зависимости от размера входных данных n .

Пример. Алгоритм суммирования всех элементов массива из n элементов:

```
sum = 0
for i = 0 to n-1:
    sum = sum + a[i]
```

Здесь на каждой итерации выполняется одна операция сложения, итераций — n , значит:

$$T(n) = n$$

Функция $T(n)$ показывает, как растёт объём работы алгоритма при увеличении размера входа. Она и служит основой для дальнейшего сравнения алгоритмов.

3. Зачем нужна асимптотическая оценка: пример сравнения двух алгоритмов

Пусть есть два алгоритма с функциями трудоёмкости:

$$T_1(n) = n^3 + 15n^2 + 72$$

$$T_2(n) = 550000n^2 + 352n + 4$$

На первый взгляд, у $T_2(n)$ огромный коэффициент 550000, а у $T_1(n)$ — маленькие коэффициенты, поэтому кажется, что T_1 всегда быстрее. Но если посчитать значения при разных n :

n	$T_1(n) = n^3 + 15n^2 + 72$	$T_2(n) = 550000n^2 + 352n + 4$
10	$1\,000 + 1\,500 + 72 = 2\,572$	$55\,000\,000 + 3\,520 + 4 \approx 5.5 \cdot 10^7$
100	$1\,000\,000 + 150\,000 + 72 \approx 1.15 \cdot 10^6$	$5\,500\,000\,000 + \dots \approx 5.5 \cdot 10^9$
1 000	$\approx 1.015 \cdot 10^9$	$\approx 5.5 \cdot 10^{11}$
10 000	$\approx 1.0015 \cdot 10^{12}$	$\approx 5.5 \cdot 10^{13}$
600 000	$\approx 2.16 \cdot 10^{17}$	$\approx 1.98 \cdot 10^{17}$
10^7	10^{21}	$\approx 5.5 \cdot 10^{16}$

При небольших и средних n алгоритм с $T_2(n)$ работает дольше (за счёт огромной константы 550000). Но при $n \rightarrow \infty$ кубическая составляющая n^3 у $T_1(n)$ рано или поздно «обгоняет» квадратичную $550000n^2$ у $T_2(n)$ — потому что степень растёт быстрее, чем можно компенсировать любой константой.

Вывод: при $n \rightarrow \infty$ поведение функции определяется её **старшим (доминирующим) слагаемым**, а все константы и слагаемые более низкого порядка становятся незначительными. Именно это наблюдение и обосновывает переход к **асимптотическим оценкам** — оценкам роста функции, которые описывают её поведение «в пределе», отбрасывая константы и младшие члены.

4. Асимптотические обозначения

Чтобы формально сравнивать алгоритмы по их скорости роста, а не по конкретным числам, вводятся три основных обозначения: O , Ω и Θ .

4.1. Оценка сверху: O (« O большое»)

Определение. Функция $f(n)$ принадлежит классу $O(g(n))$, если:

$$\exists c > 0, \exists n_0 > 0 : \forall n > n_0 : 0 \leq f(n) \leq c \cdot g(n)$$

Пишут: $f(n) \in O(g(n))$.

Смысл. Начиная с некоторого n_0 , функция $f(n)$ никогда не превышает функцию $g(n)$, умноженную на какую-то константу c . То есть $g(n)$ с точностью до постоянного множителя является **верхней границей** роста $f(n)$ — алгоритм с трудоёмкостью $f(n)$ растёт «не быстрее», чем $g(n)$.

Графически это означает следующее: если построить графики $f(n)$ и $c \cdot g(n)$, то начиная с точки n_0 график $c \cdot g(n)$ будет идти **выше или на уровне** графика $f(n)$ и оставаться выше него для всех последующих n — то есть $c \cdot g(n)$ образует «потолок» для $f(n)$.

Пример. Пусть $f(n) = 3n + 5$. Покажем, что $f(n) \in O(n)$.

Возьмём $c = 4$. Нужно найти n_0 , такое что при всех $n > n_0$ выполняется $3n + 5 \leq 4n$.

$$3n + 5 \leq 4n \iff 5 \leq n$$

Значит, при $n_0 = 5$ (и любом $n > 5$) неравенство выполняется. Следовательно, $f(n) = 3n + 5 \in O(n)$ — при этом можно было взять и другую пару (c, n_0) , доказательство неединственно, важен сам факт существования хотя бы одной такой пары.

4.2. Оценка снизу: Ω («Омега большое»)

Определение. Функция $f(n)$ принадлежит классу $\Omega(g(n))$, если:

$$\exists c > 0, \exists n_0 > 0 : \forall n > n_0 : 0 \leq c \cdot g(n) \leq f(n)$$

Пишут: $f(n) \in \Omega(g(n))$.

Смысл. Начиная с некоторого n_0 , функция $f(n)$ никогда не опускается ниже $g(n)$, умноженной на константу c . То есть $g(n)$ является **нижней границей** роста $f(n)$ — алгоритм растёт «не медленнее», чем $g(n)$.

Графически: график $c \cdot g(n)$, начиная с n_0 , идёт **ниже или на уровне** графика $f(n)$ — то есть служит «полом», ниже которого $f(n)$ не опускается.

O и Ω — взаимно двойственные понятия: если $f(n) \in O(g(n))$, то это равносильно тому, что $g(n) \in \Omega(f(n))$.

4.3. Точная асимптотическая оценка: Θ («Тета большое»)

Определение.

$$\exists c_1 > 0, c_2 > 0, n_0 > 0 : \forall n > n_0 : 0 \leq c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$$

Пишут: $f(n) \in \Theta(g(n))$.

Смысл. Это объединение O и Ω : функция $f(n)$ «зажата» между двумя вариантами одной и той же функции $g(n)$, отличающимися только константами c_1 и c_2 . Это означает, что $f(n)$ растёт **с той же скоростью**, что и $g(n)$ — не быстрее и не медленнее, с точностью до постоянных множителей.

Графически: график $f(n)$, начиная с n_0 , всё время находится в «коридоре» между графиками $c_2 \cdot g(n)$ (снизу) и $c_1 \cdot g(n)$ (сверху).

Зачем это всё нужно. Асимптотические оценки позволяют:

1. Сравнивать алгоритмы независимо от конкретного «железа», языка программирования или мелких деталей реализации — важна лишь тенденция роста при увеличении размера входа.
2. Не тратить силы на точный подсчёт констант — они всё равно теряют значение при больших n (как в примере с $T_1(n)$ и $T_2(n)$ выше).
3. Предсказывать поведение алгоритма на больших данных, даже не имея возможности его протестировать на них напрямую.

На графике сравнения функций $O(1)$, $O(\sqrt{n})$, $O(n)$ и $O(n^2)$ выше хорошо видно: даже если у более «медленной» по порядку роста функции (например, $O(n)$) в начале значения могут казаться сравнимыми с более «быстрой» ($O(n^2)$), при увеличении n разница становится колоссальной — квадратичная функция обгоняет линейную кратно и без возврата.

5. Примеры вычисления сложности циклов

Разберём типовые конструкции циклов и определим их сложность.

5.1. Цикл с фиксированным числом итераций

```
for (i = 0; i < 100; ++i) { }
```

Число итераций — **всегда 100**, независимо от размера входных данных n . Раз количество операций не зависит от n — это **константная** сложность:

$$T(n) = O(1)$$

5.2. Цикл, зависящий от размера входа

```
for (i = 0; i < n; ++i) { }
```

Цикл выполняется ровно n раз — количество операций растёт линейно вместе с размером входа:

$$T(n) = O(n)$$

5.3. Вложенный цикл, где внешний цикл — константный

```
for (i = 0; i < 100; ++i) {  
    for (i = 0; i < n; ++i) { }  
}
```

Внутренний цикл выполняется n раз, а внешний — фиксированные 100 раз. Итоговое число операций примерно $100 \cdot n$. Но 100 — это константа, не зависящая от n , поэтому при переходе к асимптотической записи она отбрасывается:

$$T(n) = 100n \implies T(n) = O(n)$$

Это важный момент: **вложенность циклов сама по себе не увеличивает порядок роста**, если один из циклов не зависит от n — растёт только тот множитель, который связан с n .

5.4. Цикл с условием, зависящим от квадрата счётчика

```
for (i = 0; i * i < n; ++i) { }
```

Цикл продолжается, пока $i^2 < n$, то есть пока $i < \sqrt{n}$. Значит, число итераций примерно равно $\sqrt{n}$:

$$T(n) = O(\sqrt{n})$$

Это классический пример сублинейной, но не константной сложности — растёт медленнее, чем n , но быстрее, чем константа.

6. Свойства асимптотических обозначений

6.1. Транзитивность

$$f(n) \in O(g(n)) \text{ и } g(n) \in O(h(n)) \implies f(n) \in O(h(n))$$

Смысл: если f растёт не быстрее g , а g растёт не быстрее h , то и f растёт не быстрее h . Свойство позволяет строить «цепочки» сравнений алгоритмов, не сравнивая каждую пару напрямую.

(Аналогичная транзитивность выполняется и для Ω , и для Θ .)

6.2. Рефлексивность

$$f(n) \in O(f(n)), \quad f(n) \in \Omega(f(n)), \quad f(n) \in \Theta(f(n))$$

Смысл: любая функция асимптотически ограничена сама собой сверху, снизу и точно — это следует напрямую из определений (достаточно взять $c = 1$).

6.3. Симметрия

$$f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$$

Смысл: отношение «растут с одинаковой скоростью» — взаимно: если f растёт как g , то и g растёт как f . Это свойство характерно именно для Θ (для O и Ω симметрии нет — они задают направленное отношение «не быстрее» / «не медленнее», а не «равенство скоростей»).

7. Сортировка вставками (Insertion Sort)

Идея алгоритма. Массив мысленно делится на две части: «отсортированную» (в начале) и «неотсортированную» (оставшаяся часть). На каждом шаге берётся первый элемент из неотсортированной части и вставляется на нужное место в отсортированной части, сдвигая при необходимости элементы вправо.

Псевдокод:

```
for i = 1 to n-1:
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key:
        a[j+1] = a[j]
        j = j - 1
    a[j+1] = key
```

7.1. Вывод оценки сложности

Рассмотрим худший случай — массив отсортирован в обратном порядке. Тогда на шаге i (при вставке i -го элемента) придётся сдвинуть все i элементов отсортированной части, то есть выполнить порядка i операций сравнения/сдвига.

Суммарное число операций по всем шагам от $i = 1$ до $i = n - 1$:

$$T(n) = \sum_{i=1}^{n-1} i = 1 + 2 + \dots + (n-1) = \frac{(n-1) \cdot n}{2} = \frac{n^2 - n}{2}$$

7.2. Почему константы отбрасываются

Получили точную формулу:

$$T(n) = \frac{n^2 - n}{2}$$

Чтобы перейти к асимптотической записи $O(n^2)$, нужно показать, что $T(n) \in O(n^2)$ по определению из раздела 4.1, то есть найти $c > 0$ и $n_0 > 0$, такие что при всех $n > n_0$:

$$\frac{n^2 - n}{2} \leq c \cdot n^2$$

Возьмём, например, $c = 1$. Тогда:

$$\frac{n^2 - n}{2} \leq n^2 \iff n^2 - n \leq 2n^2 \iff -n \leq n^2$$

Последнее неравенство верно для всех $n \geq 0$ (при $n > 0$ правая часть положительна, левая отрицательна). Значит, уже при $n_0 = 0$ (или любом положительном n_0) неравенство выполняется для всех $n > n_0$.

Таким образом, формально доказано:

$$\frac{n^2 - n}{2} \in O(n^2)$$

Интуитивно это отбрасывание константы и младшего члена объясняется так же, как и в примере из раздела 3: при $n \rightarrow \infty$ слагаемое n^2 растёт значительно быстрее, чем линейное слагаемое $-n$, и делитель $\frac{1}{2}$ — это просто постоянный множитель, который не меняет *порядок* роста функции (он входит в константу c из определения O). Поэтому итоговая асимптотическая оценка сортировки вставками:

$$T(n) = O(n^2)$$

7.3. Границы (лучший и худший случай)

- **Худший случай** (массив отсортирован в обратном порядке): каждый элемент нужно сдвигать через всю отсортированную часть — $T(n) = O(n^2)$.
- **Лучший случай** (массив уже отсортирован): на каждом шаге достаточно одного сравнения, сдвигов не требуется — $T(n) = O(n)$.

7.4. Оценка по памяти

Для памяти действует похожий принцип, что и для времени: важно, зависит ли объём **дополнительной** (сверх исходного массива) памяти от размера входа n или нет.

- Если алгоритму для работы требуется **константный** объём дополнительной памяти (не зависящий от n) — за этим специально следить не нужно, так как асимптотически это не влияет на оценку.
- Если же дополнительная память **растёт вместе с n** (то есть нужно что-то дополнительно хранить пропорционально размеру входа) — за этим нужно следить и включать в оценку сложности по памяти.

Сортировка вставками работает «на месте» (in-place) — она переставляет элементы внутри исходного массива и не создаёт дополнительных структур, растущих вместе с n , поэтому её память константная.